

Invoice Extraction Pipeline

Design document — as built (Django + Celery + Redis + Postgres + pdfplumber)

Overview

A Django app that accepts invoice PDF uploads and processes them asynchronously. A POST request writes the file to disk and a job row to Postgres, then hands the job off to a Celery worker over a Redis-backed queue so the HTTP request returns immediately instead of blocking on parsing. The worker reads the PDF with pdfplumber, pulls out vendor, invoice, and line-item data using keyword and regex matching, and writes the structured result back to Postgres.

This document describes the system as it exists in the codebase today, not the original aspirational design. Where the two diverge, it says so explicitly — see "Known limitations" below. That's a deliberate choice: an honest account of what's built, what's stubbed, and what would break in production is more useful, and more credible to a reviewer, than a polished description of features that don't exist yet.

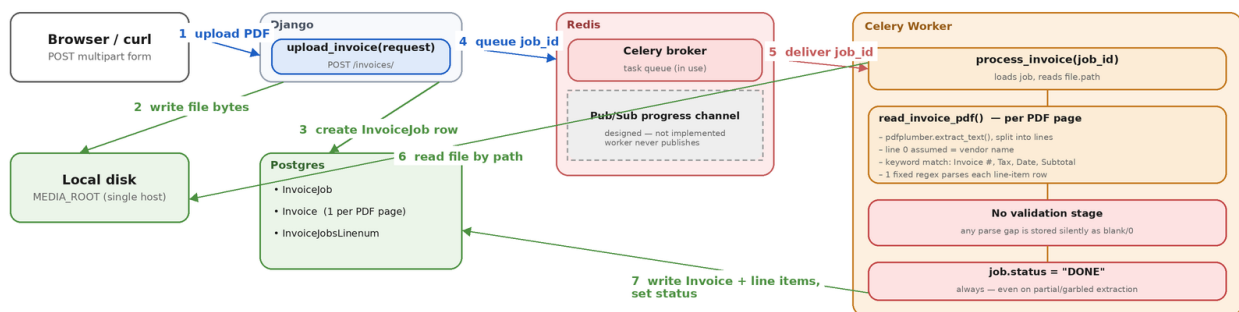
Tech stack

Django (web framework, ORM, admin) · Celery (async task queue) · Redis (Celery's message broker) · PostgreSQL (system of record) · pdfplumber (PDF text extraction) · re (rule-based field/line-item parsing)

Architecture

Invoice Extraction Pipeline — as built

Django + Celery + Redis (broker only) + Postgres + pdfplumber



Response to browser (job_id + status=PENDING) returns immediately after step 4 — client has no way to learn the outcome afterward except polling Postgres/admin directly; there is no status endpoint yet.

Known gaps vs. the original design (see "Known limitations" in the write-up):

No live progress — the Pub/Sub channel above was designed but the worker never publishes to it.
No error handling in the task — an exception (bad layout, failed regex) marks the Celery task FAILED, but the InvoiceJob row is never updated, so it stays "PENDING" in Postgres forever with no failure_reason set.
Multi-page PDFs raise IntegrityError — InvoiceJob is a OneToOneField, but read_invoice_pdf() creates one Invoice per page, so page 2 of any multi-page invoice violates the unique constraint.

Numbered arrows follow one request through the system. The grayed, dashed box inside Redis is not a running feature — it's where a Pub/Sub progress channel was designed to go; nothing in the codebase publishes to it today.

Request flow

1–3. Upload. A client POSTs multipart form data to `POST /invoices/` (`invoice_jobs/views.py`, `upload_invoice`). The view creates an `InvoiceJob` row (status defaults to `PENDING`), which via its `FileField` writes the uploaded PDF to `MEDIA_ROOT/invoices/%Y/%m/` on local disk.

4. Queue. The view calls `process_invoice.delay(str(job.id))`. Only the job's UUID goes over the wire to Redis — never the file bytes. The view then returns `{"job_id": ..., "status": "PENDING"}` with a 201, without waiting for the task.

5. Dispatch. A separate, already-running Celery worker process (started with `celery -A invoices worker`) picks the task up off the Redis queue and invokes `process_invoice(job_id)` in `invoice_jobs/tasks.py`.

6–7. Extract and write. The task loads the `InvoiceJob`, resolves `job.file.path`, and calls `read_invoice_pdf()`, which opens the PDF with `pdfplumber` and, for each page, extracts text, parses it into a vendor name, invoice metadata, and line items, and writes an `Invoice` row plus its `InvoiceJobsLinenum` rows to Postgres. The task then unconditionally sets `job.status = "DONE"`.

Extraction logic

`read_invoice_pdf()` runs per page of the PDF and treats each page's text as a flat list of lines, matched against a handful of fixed rules:

- Line 0 of the page is always treated as the vendor name.
- A line containing "Description" opens the line-item table; a line containing "Subtotal" closes it. Every line in between is matched against one fixed regex (`description`, `qty`, `$unit_price`, `$line_total`) and, if it matches, becomes one `InvoiceJobsLinenum` row.
- Lines containing "Invoice #", "Tax", "Invoice Date" and "Subtotal" are split on `:` to pull out those specific fields. The date is parsed with a hardcoded format string (`%m/%d/%Y`).
- `total` is never read from the PDF — it's computed in code as `subtotal + tax`.

In short: this is a single hardcoded invoice layout, not a general rule engine with multiple templates. It will parse the layout it was written against and silently mis-parse or crash on anything else.

Data model (as built)

```

class InvoiceJob(models.Model):
    id = models.UUIDField(default=uuid.uuid4, primary_key=True)
    file = models.FileField(upload_to="invoices/%Y/%m/")
    status = models.CharField(default="PENDING", max_length=20) # free text, no choices
    failure_reason = models.TextField(blank=True) # defined, never written to
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

class Invoice(models.Model):
    job = models.OneToOneField(InvoiceJob, on_delete=models.CASCADE)
    invoice_number = models.CharField(max_length=100, null=True, blank=True)
    vendor_name = models.CharField(max_length=255, null=True, blank=True)
    invoice_date = models.DateField(null=True, blank=True)
    subtotal = models.DecimalField(max_digits=10, decimal_places=2)
    tax = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)
    total = models.DecimalField(max_digits=10, decimal_places=2)

class InvoiceJobsLinenum(models.Model):
    invoice = models.ForeignKey(Invoice, on_delete=models.CASCADE)
    description = models.CharField(max_length=500)
    quantity = models.DecimalField(max_digits=10, decimal_places=2, default=1)
    unit_price = models.DecimalField(max_digits=12, decimal_places=2)
    line_total = models.DecimalField(max_digits=12, decimal_places=2)

```

All three models are registered in `admin.py`, so job status, extracted invoices, and line items can all be inspected through the Django admin without writing a single query — the closest thing the project currently has to a status UI.

Known limitations

These are gaps against the original design brief and issues visible directly in the current code. Listed honestly because they're also exactly the kind of thing a reviewer will find by reading the source, and because each one has a concrete, describable fix:

No live progress. The original design called for the worker to publish staged progress events to a Redis Pub/Sub channel (`job:{id}:progress`), relayed to the browser over Server-Sent Events. Neither side exists yet — Redis is only doing broker duty. A client that wants to know a job's outcome today has to poll Postgres or check the admin directly; there's no status endpoint.

Task failures are invisible. `process_invoice` has no try/except. If pdfplumber can't open the file, or a line fails the line-item regex, the task raises, Celery marks it FAILED in its own internal bookkeeping — but the `InvoiceJob` row is never touched. It stays "PENDING" in Postgres forever, and `failure_reason` (which exists on the model for exactly this) never gets set.

Multi-page PDFs will crash. `Invoice.job` is a `OneToOneField`, but `read_invoice_pdf()` loops per page and calls `Invoice.objects.create()` on every page. A second page on the same job means a second `Invoice` pointing at the same job id, which violates the one-to-one constraint and raises an `IntegrityError`. Only single-page invoices work today.

One hardcoded layout, no validation. The parser assumes a fixed structure (vendor name on line 1, specific keywords, one line-item regex) with no fallback and no post-extraction sanity check. A differently formatted invoice won't be flagged for review — it'll either crash or silently save blank/incorrect fields, since `invoice_number`, `vendor_name` etc. are all nullable.

Smaller gaps: the upload view is `@csrf_exempt` with no auth, `MEDIA_ROOT` is a local path on one machine (no shared/object storage), there's no rate limiting, no automated tests (`tests.py` is still the generated stub), and there's no Docker Compose setup, so running it requires Postgres, Redis, the Django dev server, and a Celery worker all started by hand.

Suggested next steps

Make failures visible. Wrap `process_invoice` in `try/except`; on failure, set `status="FAILED"` and write the exception into `failure_reason` instead of leaving the row stuck on `PENDING`.

Fix the multi-page bug. Aggregate extracted text across all pages before creating a single `Invoice` per job, rather than one per page.

Add a validation/NEEDS_REVIEW pass. After extraction, check that line items sum to the stated total and that required fields were found; if not, flag the job instead of marking it `DONE`.

Give status choices real enforcement. Restrict status to an explicit `choices=` list (`PENDING` / `PROCESSING` / `NEEDS_REVIEW` / `DONE` / `FAILED`) so invalid states can't be written by accident.

Live progress. Add the Pub/Sub publish side in the worker and an SSE view in Django, per the original design, once the above is solid.

Package it. A Docker Compose file (web, worker, redis, postgres) and a couple of `pytest` cases around the regex parser would make this reviewable end-to-end without manual setup.

For the portfolio card

Tags: Python, Django, Celery, Redis, PostgreSQL, pdfplumber, Async processing

Suggested repo name: `invoice-extraction-pipeline`

One-line summary (draft): "A Django + Celery pipeline that processes invoice PDFs asynchronously — uploads return immediately while a background worker extracts structured vendor, invoice, and line-item data with pdfplumber and rule-based parsing, persisted to Postgres."

That summary describes the plumbing and extraction that actually work today. Save any mention of live progress or a review queue for after those are built — claiming them now is the one framing choice this document argues against.